
AN0020-EC NB-IoT SDK Compiling Guidebook

_V1.0

EigenCOMM Wireless Microcontroller

Document Description

The present document guides user to configure the Makefile and compile the EC NB-IoT SDK for secondary development.

Contents

1.	Foreword	4
1.1	Scope	4
1.2	Main content.....	4
2.	Building the SDK compilation environment	4
2.1	How to compile SDK in Windows	4
2.1.1	Setup a cross compiler	4
2.1.2	Compilation step.....	5
2.1.3	Compile output	7
2.1.4	Analyze the compilation results.....	9
3.	SDK compilation environment.....	9
3.1	Makefile structure.....	10
3.2	Makefile.inc.....	10
3.3	Makefile.....	12
3.4	Custom compliation.....	12
3.5	How to add own project.....	14
4.	Appendix	16
4.1	Example of compilation lwm2m_example project	16
5.	Version.....	16
6.	About US	17

Figure Index

Figure1.change the configure of KeilBuild.bat	5
Figure 2. KeilBuild.bat help	7
Figure 3. Compile result	8
Figure 4. Out files.....	8
Figure 5. Map file analysis	9
Figure 6. Main structure	10
Figure 7. Build configuration	14
Figure 8. Add new project configuration.....	15
Figure 9. unilog not define	16

Table Index

Table 1. All supported commands	6
---------------------------------------	---

1. Foreword

1.1 Scope

The present document helps users quickly understand the SDK compilation and debugging environment.

1.2 Main content

1. How to build the SDK compilation environment.
2. How to compile sample projects using EC NB-IoT SDK.
3. How to add your own project.

2. Building the SDK compilation environment

After unpack the SDK, There are several batch files in root directory. For example, KeilBuild.bat is a One-Key compilation script that supports the ARMCC toolchain.

2.1 How to compile SDK in Windows

2.1.1 Setup a cross compiler

To ARMCC, user need install keil_v5. The process of installing keil_v5 will automatically install ARMCC. If keil_v5 is already installed in the root of the C drive. The path of the ARMCC need update in KeilBuild.bat, as shown below.

```

1 @echo off
2 @echo %PATH% | findstr /c:"%~dp0tools/msys64/usr/bin">nul
3 @if %errorlevel% equ 1 set PATH=%~dp0tools/msys64/usr/bin;%PATH%
4 rem @set PATH=%~dp0tools/msys64/usr/bin;%PATH%
5
6 set PROJECT_NAME=at_command
7 set BOARD_NAME=ec616_0h00
8
9
10 set KEILCC_PATH="C:/Keil_v5/ARM/ARMCC/bin/"
11 set PROTOCOL_TOOL_DB_PATH="..\PROTOCOL\Tool\UnilogPsParser\DB"
12 set COMMON_TOOL_DB_PATH="..\TOOLS\UniLogParser\DB"
13
14 echo build.bat version 20180330
15 echo KEILCC_PATH: %KEILCC_PATH%
16
17 if not %KEILCC_PATH% == "" (
18     if not exist %KEILCC_PATH% (
19         echo ERROR: Please check KEILCC_PATH setting, exit!!!
20         goto end
21     )
22 ) else (
23     echo ERROR: Please set KEILCC_PATH firstly, exit!!!
24     goto end
25 )
26

```

Figure 1.change the configure of KeilBuild.bat

2.1.2 Compilation step

1. Double click cmd.exe in the root directory and enter the Command Prompt.
2. Run KeilBuild.bat, this batch file supports different parameters, and the parameter is optional. If the user run KeilBuild.bat without parameter, default project will be compiled. Otherwise, the specific project will compile according to specified parameter.

The format of parameter is: #KeilBuild.bat [board name]-[project name]-[option]

The following table takes board ec616 as an example.

parameter	content	description	
Board name	ec616_0h00	EC616 board name	
Project name	at_command	Example project of AT command	Default project
	bootloader	Example project of bootloader	
	driver_example	Example project of driver	
	fs_example	Example project of file system	
	onenet_example	Example project of OneNET	
	Aliyun_example	Example project of aliyun	
	socket example	Example project of socket	
	mqtt_example	Example project of MQTT	
	coap_example	Example project of CoAP	
	lwmm2m_example	Example project of LWM2M	

Option	clean	clean current build project	
	clall	clean all projects have built	
	unilog	before compile create debug_log.h	
	doc	create document	
	verbose	output detailed help information during compilation	
	help	output help information	
	list	list all board name and project name support	
	allprojects	compile all the projects	
	defaultall	compile default project and bootloader project	

Table 1. All supported commands

Use Keilbuild.bat help to see all supported command parameters and usage, as shown below.

```

D:\WorkArea\yjzhong\newA0\PLAT>KeilBuild.bat help
Keilbuild.bat version 20180330
KEILCC_PATH: "C:/Keil_v5/ARM/ARMCC/bin/"
OPTION: help
=====
-----Show How to Build Project-----
=====
Keilbuild.bat Board-Project-Option
Board,Project,Option could be omitted, if so default Board and project will be used
=====
Supported Options:
doc----run doxygen to generate documents
unilog---pre-parse the log before compile
clean----clean the build output files
clall----clean all output files for every board and project
jekins_release---trigger jekins_release
allprojects---trigger compile all example projects under default board
list---list current supported boards and supported project for each board
help---show how to use the build script

Example start -- Example start -- Example start -- Example start
-----
Keilbuild.bat ec616_0h00-driver_example"
build driver_example project for board ec616_0h00"
=====
Keilbuild.bat ec616_0h00-bootloader"
build bootloader project for board ec616_0h00"
=====
Keilbuild.bat ec616_0h00-driver_example-clean"
clean driver_example project for board ec616_0h00"
=====
Keilbuild.bat clean"
clean default project for default board
=====
Keilbuild.bat clall"
clean all output files for every board and project
=====
Keilbuild.bat unilog"
perform log preparse of default project for default board
=====
Keilbuild.bat list"
list current supported boards and supported project for each board
-----
Example end -- Example end -- Example end -- Example end
D:\WorkArea\yjzhong\newA0\PLAT>

```

Figure 2. KeilBuild.bat help

2.1.3 Compile output

After compilation, the terminal will output the compilation result. Program size indicates the memory usage.

[illegible]

Figure 3. Compile result

The intermediate files, library files, axf/hex files, map files, and corresponding disassembly files will be output to different subdirectories according to different board and project configurations, for example:

PLAT\out\ec616 0h00\at command

the out files as shown below.

FIRMWARE	2020/6/4 18:31	
lib	2020/6/12 14:01	
PLAT	2020/6/4 18:30	
PROTOCOL	2020/6/4 18:30	
app-demo-flash.axf	2020/6/12 14:01	24,838 KB
app-demo-flash.bin	2020/6/12 14:01	1,859 KB
app-demo-flash.hex	2020/6/12 14:01	5,228 KB
app-demo-flash.htm	2020/6/12 14:01	10,132 KB
app-demo-flash.map	2020/6/12 14:01	3,963 KB
app-demo-flash.symbols	2020/6/12 14:01	311 KB
app-demo-flash.txt	2020/6/12 14:01	47,539 KB
comdb.txt	2020/6/12 14:01	967 KB

Figure 4. Out files

The log of the compilation process is `outbuildinglog.txt` in the `SDK/out`, for example:

PLAT\out\ec616 0h00\outbuildlog.txt

2.1.4 Analyze the compilation results

To ARMCC, user can get the image details from \$(BUILDDIR)/\$(BINNAME).map. The map contains the address and the size of each load region, execution region, and input section in the image, including linker-generated input sections. It also contains the RAM/FLASH usage statistics. In addition, user can use this information to optimize the code.

900918	206492	85472	13132	200796	5061684	Library Totals
468	0	205	42	5	0	(incl. Padding)

Code (inc. data)	RO Data	RW Data	ZI Data	Debug	Library Name	
7526	742	696	45	804	223214	libdriver.lib
23632	4948	777	140	66008	276210	libfreertos.lib
111508	8832	20555	1250	66636	1138577	libfw.lib
76360	13590	2586	239	18752	529492	libmiddleware_3dparty.lib
48718	8242	1815	2451	27889	408551	libmiddleware_ec.lib
629836	170020	58705	8953	20702	2479494	libps.lib
132	36	0	0	0	3154	libstartup.lib
2248	82	133	12	0	2568	mc_w.l
490	0	0	0	0	424	mf_w.l

900918	206492	85472	13132	200796	5061684	Library Totals

=====						
Code (inc. data)	RO Data	RW Data	ZI Data	Debug		
902288	206578	85552	13132	201820	4896687	Grand Totals
902288	206578	85552	13132	201820	4896687	ELF Image Totals
902288	206578	85552	13132	0	0	ROM Totals
=====						
Total RO	Size (Code + RO Data)	987840 (964.69kB)				
Total RW	Size (RW Data + ZI Data)	214952 (209.91kB)				
Total ROM	Size (Code + RO Data + RW Data)	1000972 (977.51kB)				
=====						

Figure 5. Map file analysis

3. SDK compilation environment

This chapter will further introduce EC NB-IoT SDK compilation details, guide users how to use the SDK's compilation rules, how to modify the corresponding Makefile to meet different compilation requirements when add code.

3.1 Makefile structure

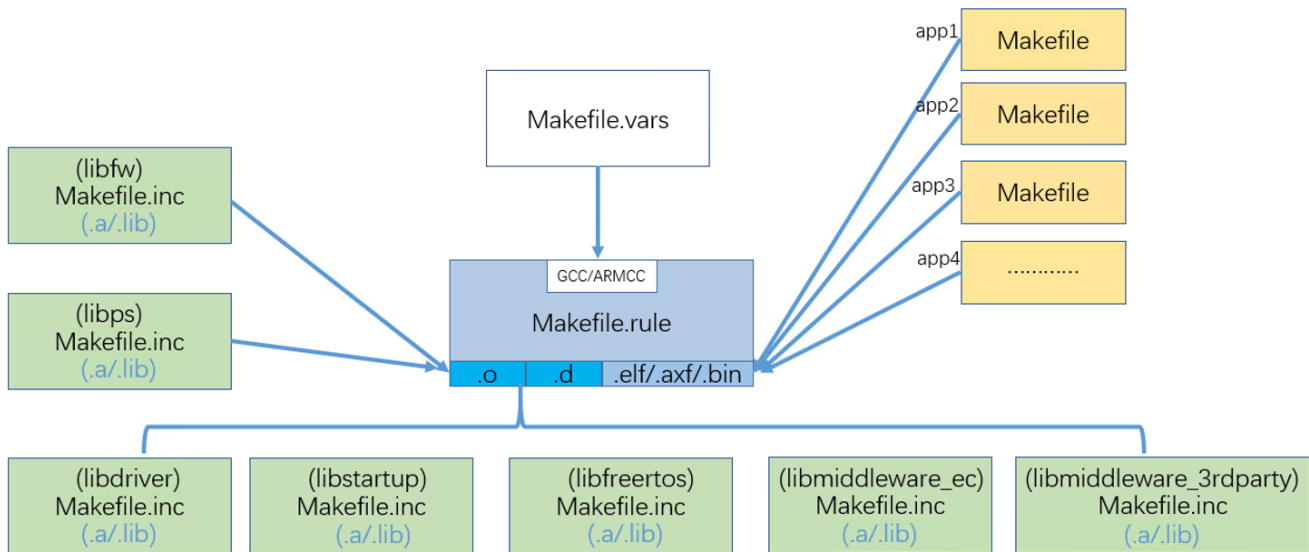


Figure 6. Main structure

As shown in Figure 9, Makefile.rule is the core of the EC NB-IoT SDK compilation component, and it support both ARMCC and GCC. It defines the generation rules for common .o object files and dependent files .d file. It also supports incremental compilation. Makefile.var is abstract some compilation variables used by Makefile.rule. Different Makefile.inc are distribute in different directories to be compile. All These Makefile.inc will link all object files in the SDK, and the Makefile at the application layer will indicate the final bin file.

3.2 Makefile.inc

This chapter use SDK\middleware\thirdparty\wakaama\Makefile.inc as an example to explain how to write Makefile.inc to compile a new module. User can refer to this file to write Makefile.inc when adding a new module.

//root path of source file

LWM2M_DIR := \$(TOP)/PLAT/middleware/thirdparty/wakaama

//CFLAGS compile options that this module depends on

CFLAGS += -DLWM2M_OLD_CONTENT_FORMAT_SUPPORT -DNO_DECLTYPE -DLWM2M_BOOTSTRAP -DWITH_LOGS -DLWM2M_WITH_LOGS

//header file path that this module depends on

*CFLAGS_INC += -I \$(LWM2M_DIR)/core \
-I \$(LWM2M_DIR)/core/er-coap-13 \
-I \$(LWM2M_DIR)/examples/client \
-I \$(LWM2M_DIR)/examples/shared*

//source file path that this module depends on

*LWM2M_SRC_DIRS += \$(LWM2M_DIR)/core \
\$(LWM2M_DIR)/core/er-coap-13 \
\$(LWM2M_DIR)/examples/client*

\$(LWM2M_DIR)/examples/shared

//need to exclude compiled files

LWM2M_EXCLUDE_FILES :=

//search all source files

LWM2M_CSRC = \$(foreach dir, \$(LWM2M_SRC_DIRS), \$(wildcard \$(dir)/.c))*

//Eliminate files that don't need to be compiled

LWM2M_CFILES = \$(filter-out \$(LWM2M_EXCLUDE_FILES), \$(LWM2M_CSRC))

//.c to .o

LWM2M_COBJSTEMP := \$(patsubst %.c, %.o, \$(LWM2M_CFILES))

//Absolute path to relative path

LWM2M_COBJSTEMP:=\$(subst\$(LWM2M_DIR),PLAT/middleware/thirdparty/wakaama,\$(LWM2M_COBJSTEMP))

//Prefix to \$(BUILDDIR) path

LWM2M_COBJS := \$(addprefix \$(BUILDDIR)/, \$(LWM2M_COBJSTEMP))

//Include dependent file

-include \$(LWM2M_COBJS:.o=.d)

//GCC compilation branch

ifeq (\$(TOOLCHAIN),GCC)

//include GCC special header file

CFLAGS_INC += -I \$(TOP)/PLAT/os/freertos/portable/gcc

//Declare to generate .a lib library

lib-y += liblwm2m.a

//GCC library compilation rules

\$(BUILDDIR)/lib/liblwm2m.a: \$(LWM2M_COBJS)

@mkdir -p \$(dir \$@)

\$(ECHO) AR \$@

\$(Q)\$ (AR) -cr \$@ \$^

endif

//ARMCC compilation branch

ifeq (\$(TOOLCHAIN),ARMCC)

//include ARMCC special header file

CFLAGS_INC += -I \$(TOP)/PLAT/os/freertos/portable/keil

//Declare to generate .a lib library

lib-y += liblwm2m.lib

//ARMCC library compilation rules

```
$(BUILDDIR)/lib/liblwm2m.lib: $(LWM2M_COBJS)
    @mkdir -p $(dir $@)
    $(ECHO) AR $@
    $(Q)$ (AR) $(ARFLAGS) $@ $^
endif
```

3.3 Makefile

This chapter use SDK\project\ec616_0h00\apps\onenet_example\ARMCC as an example to explain how to compile a new project. User can refer to this file to add a new project.

```
// declare board-level goals supported by the project
AVAILABLE_TARGETS = ec616_0h0

//choose the toolchain
TOOLCHAIN = ARMCC

//define the name of the BIN
BINNAME = onenet-demo-flash

//define the TOP directory
TOP := ../../../../..

//define build options
BUILD_AT = n
BUILD_AT_DEBUG = n
THIRDPARTY_CISONENET_ENABLE = y
THIRDPARTY_WAKAAMA_ENABLE = n
// header file path
CFLAGS_INC += -I ../inc
// source file
obj-y += PLAT/project/$(TARGET)/apps/onenet_example/src/app.o \
        PLAT/project/$(TARGET)/apps/onenet_example/src/cisAsynEntry.o \
        PLAT/project/$(TARGET)/apps/onenet_example/src/bsp_custom.o
//include compilation rules
include $(TOP)/PLAT/tools/scripts/Makefile.rules
```

3.4 Custom compilation

The EC NB-IoT SDK contains a lot of self-developed software and open source software. This SDK has undergone modular processing, and the compilation coupling between each other is reduce, thereby facilitating user to perform highly granular custom compilation according to requirements, thereby greatly improving the compilation efficiency of the code. The highly granular customization features are reflect in all peripheral drivers of the EC NB-IoT, open source software and protocol stack software, and the SDK supports custom compilation.

The EC NB-IoT SDK organizes a set of default module custom compilation configurations according to different application scenarios. If the user does not make new customizations to the custom compilation options in the Makefile of his project, the compilation system will use default module customize the compilation

configuration to compile. Users can view all supported custom compilation options of the EC NB-IoT SDK through `ecxxx_0h00.mk` in the `SDK\device\eigencomm\board\ecxxx_0h00`.

Whether to support the AT command, whether to compile the dual-chip solution or the OpenCPU solution, is mainly achieved by configuring the following options:

`BUILD_AT ?= y`

`BUILD_AT_DEBUG ?= y`

For the compilation of bootloader project, you do not need to enable the compilation of the OS code. You can control this with the following option:

`BUILD_OS ?= y`

For peripheral drivers, the probability of enabling compilation is usually large, so compilation is enabled in the default compilation configuration. as follows:

`DRIVER_USART_ENABLE ?= y`

`DRIVER_SPI_ENABLE ?= y`

`DRIVER_I2C_ENABLE ?= y`

`DRIVER_DMA_ENABLE ?= y`

`DRIVER_CACHE_ENABLE ?= y`

`DRIVER_PMU_ENABLE ?= y`

`DRIVER_FLASHRT_ENABLE ?= y`

`DRIVER_FLASH_ENABLE ?= y`

`DRIVER_GPIO_ENABLE ?= y`

`DRIVER_GPR_ENABLE ?= y`

`DRIVER_TIMER_ENABLE ?= y`

`DRIVER_UNILOG_ENABLE ?= y`

`DRIVER_WDG_ENABLE ?= y`

`DRIVER_IC_ENABLE ?= y`

`DRIVER_CIPHER_ENABLE ?= y`

`DRIVER_CIPHER_FLH_ENABLE ?= y`

`DRIVER_EFUSE_ENABLE ?= y`

`DRIVER_CIPHER_TLS_ENABLE ?= y`

`DRIVER_MCUMODE_ENABLE ?= y`

`DRIVER_ADC_ENABLE ?= y`

If the user feels that a driver module is not need when evaluating application requirements, he can set the compilation option of the module to “n” in the Makefile of his project. For example, the user does not need to use the SPI and I2C peripherals after evaluation. It can modify as follows:

```

AVAILABLE_TARGETS = ec616_0h00 ec616z0_0h00
TOOLCHAIN          = ARMCC
BINNAME           = lwm2m-demo-flash
TOP               := ../../../../...

BUILD_AT          = n
BUILD_AT_DEBUG    = n
THIRDPARTY_CISONENET_ENABLE = n
THIRDPARTY_SWING_ENABLE = n
THIRDPARTY_WAKAAMA_ENABLE = y
THIRDPARTY_MBEDTLS_ENABLE = n
THIRDPARTY_TINYDTLS_ENABLE = y
DRIVER_SPI_ENABLE = n
DRIVER_I2C_ENABLE = n

CFLAGS_INC        += -I ../inc
obj-y             += PLAT/project/$(TARGET)/apps/lwm2m_example/src/app.o \
                    PLAT/project/$(TARGET)/apps/lwm2m_example/src/bsp_custom.o
include $(TOP)/PLAT/tools/scripts/Makefile.rules

```

Figure 7. Build configuration

Note: The custom default configuration option is used to indicate that if the compilation option is not defined in the Makefile, the default configuration will be used. In contrast, the use of "=" in the Makefile of the project means that the default configuration is overwritten and the newly defined configuration is enabled.

Similarly, for self-developed modules and open source modules in Middleware, the EC NB-IoT SDK also defines corresponding custom compilation options. The difference is that for such modules, compilation is usually prohibited, so most of the default compilation configuration is forbid, but there are exceptions, For example, modules such as littlefs, lwip, ping, rohc are necessary for NB devices. So these modules are compile by default.

#Choose if below thirdparty software need to be compiled

```

THIRDPARTY_CISONENET_ENABLE ?= n
THIRDPARTY_WAKAAMA_ENABLE ?= n
THIRDPARTY_CTWING1_5_ENABLE ?= n
THIRDPARTY_MQTT_ENABLE ?= n
THIRDPARTY_LIBCOAP_ENABLE ?= n
THIRDPARTY_HTTPC_ENABLE ?= n
THIRDPARTY_LITTEFS_ENABLE ?= y
THIRDPARTY_IPERF_ENABLE ?= y
THIRDPARTY_CJSON_ENABLE ?= n
THIRDPARTY_LWIP_ENABLE ?= y
THIRDPARTY_PING_ENABLE ?= y
THIRDPARTY_ROHC_ENABLE ?= y
THIRDPARTY_MBEDTLS_ENABLE ?= n
THIRDPARTY_RTT_ENABLE ?= y
THIRDPARTY_ALIIOT_ENABLE ?= n

```

3.5 How to add own project

First copy SDK\project\\$(TARGET)\templates\app_demo to SDK\project\\$(TARGET)\apps, then perform the following steps:

1. Modify the folder name app_demo to the new project name xxx

2. Add source files and header files to the src and inc directories for new applications.
3. Modify the Makefile under ARMCC directory

```
AVAILABLE_TARGETS = ecxxx_0h00
```

```
TOOLCHAIN = ARMCC
```

```
TOP := ../../../../..
```

```
// configure new custom compilation options
```

```
BUILD_AT = n
```

```
BUILD_AT_DEBUG = n
```

```
THIRDPARTY_CISONENET_ENABLE = n
```

```
THIRDPARTY_CTWING1_5_ENABLE = n
```

```
THIRDPARTY_WAKAAMA_ENABLE = y
```

```
// define new object file name
```

```
BINNAME = xxx-demo-flash
```

```
// add new compiled files
```

```
obj-y += sdk/project/$(TARGET) / apps/src/app.o \
```

```
sdk/project/$(TARGET) / apps/src/bsp_custom.o \
```

```
sdk/project/$(TARGET) / apps/src/xxx.o
```

```
include $(TOP)/sdk/tools/scripts/Makefile.rules
```

4. Modify the KeilBuild.bat/GccBuild.bat in the SDK root directory to add new configuration to add a new project, Take adding a new project lwm2m_example as an example, as shown below:

```
echo PROJECT_NAME: %PARAMETERS% | findstr "onenet_example"
if not errorlevel 1 (
    set PROJECT_NAME=onenet_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "cisdsm_example"
if not errorlevel 1 (
    set PROJECT_NAME=cisdsm_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "socket_example"
if not errorlevel 1 (
    set PROJECT_NAME=socket_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "https_example"
if not errorlevel 1 (
    set PROJECT_NAME=https_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "lwm2m_example"
if not errorlevel 1 (
    set PROJECT_NAME=lwm2m_example
)
```

Figure 8. Add new project configuration

4. Appendix

4.1 Example of compilation lwm2m_example project.

When compiling this project for the first time, you need to add the optional parameter “unilog”. It will generate a new unilog database file. If no unilog changed this optional parameter is default. If you add unilog in source file and not use the parameter “unilog”, the following error will prompt:

```

./../../../../PLAT/middleware/thirdparty/wakaama/examples/shared/connection.c: 0 warnings, 1 error
make[1]: *** [../../../../PLAT/tools/scripts/Makefile.rules:163: ../../../../../../PLAT/out/ec616_0h00/at_command/PLAT/middleware/thirdparty/wakaama/examples/shared/connection.o] Error 1
make[1]: *** Waiting for unfinished jobs....
../../../../PLAT/middleware/eigencomm/at/atcmdcustomer/src/api/atec_lwm2m_cnf_ind_api.c", line 897: Error: #20: identifier "UNILog_LWM2M_nodt1s_lwm2mConnectServer_1" is undefined

COMM_TRACE(UNILog_LWM2M, nodt1s_lwm2mConnectServer_1, P_INFO, 0, "enter");

../../../../PLAT/middleware/eigencomm/at/atcmdcustomer/src/api/atec_lwm2m_cnf_ind_api.c: 0 warnings, 1 error
make[1]: *** [../../../../PLAT/tools/scripts/Makefile.rules:163: ../../../../../../PLAT/out/ec616_0h00/at_command/PLAT/middleware/eigencomm/at/atcmdcustomer/src/api/atec_lwm2m_cnf_ind_api.o] Error 1
make[1]: Leaving directory 'C:/cygdrive/d/WorkArea/yjzhong/testA0/PLAT/project/ec616_0h00/apps/at_command/ARIMCC'
make: *** [Makefile:11: keilall] Error 2
fail
#####
##                                     ##
##          #####      ##      ##      ##      ##      ##
##          ##      ##      ##      ##      ##      ##
##          ##      ##      ##      ##      ##      ##
##          #####      ##      ##      ##      ##
##          ##      ##      ##      ##      ##      ##
##          ##      ##      ##      ##      ##      ##
##          ##      ##      ##      ##      ##      ##
#####
D:\WorkArea\yjzhong\testA0\PLAT>

```

Figure 9. unilog not define

Then you should compile again with the parameter “unilog”, like KelBuild.bat ec616-lwm2m_example-unilog

5. Version

Version	Date	Comments
V1.0	2019-12-2	
V1.1	2020-06-15	Remove the Chinese characters

6. About US

Shanghai EigenCOMM Technology Co.,Ltd is established in Feb 2017 in Zhangjiang Hi-Tech park, Shanghai China (www.eigencomm.com) . EigenCOMM focuses on cellular based IoT chipset and solution. With accumulation of experience over the years in algorithm, L1/L2/L3 protocol, SoC, RF , platform & application software as well as the communication architecture and system, team makes its way to IoT industry.

The ultra-low power NB-IoT chipset EC616(S) has already enter mass production stage. The 4G Cat.1 chipset EC618 is in engineering samples stage, 5G is also in the R&D stage.

Eigencomm Technologies Ltd.

Address: Room 707, No 298 Xiangke Road, Shanghai, China

Zip Code: 201210

Phone:

E-mail:

Website: <http://www.eigencomm.com>

Support Window

E-Mail: support@eigencomm.com